



ECOLE CENTRALE NANTES

Année Universitaire 2025-2026

Analyse Topologique des Données (TDA) appliquée au Deep Learning

Rapport Final

Auteurs :
Maël KRIMISSA
Victor SIMON

Encadrant :
Pr. Bertrand MICHEL

9 avril 2026

Table des matières

1	Introduction	3
1.1	Contexte et motivations	3
1.2	Objectifs du projet et bilan des travaux précédents	3
1.3	Évolution de la problématique : vers le Topological Deep Learning	3
1.4	Structure du rapport	3
2	Fondations Théoriques : De la Géométrie à la Topologie	4
2.1	Des complexes simpliciaux à l'homologie persistante	4
2.2	Stabilité théorique : distance de Bottleneck	5
2.3	Bilan de la preuve de concept et limites de l'approche statique	5
3	Intégration de la Topologie dans le Deep Learning	6
3.1	De l'extraction statique à l'apprentissage de bout en bout	6
3.1.1	Le problème de l'optimisation déconnectée	6
3.1.2	Vers des signatures entraînaibles	7
3.1.3	Le défi de l'invariance par permutation	7
3.2	L'architecture PersLay : une couche neuronale topologique	7
3.2.1	Cadre théorique et formulation mathématique	7
3.2.2	Unification et différentiabilité	8
3.2.3	Vers la régularisation de l'espace latent	8
3.3	Régularisation Topologique : Préserver la Continuité de l'Espace Latent	9
3.3.1	Pathologies de l'espace latent	9
3.3.2	La Topological Loss	9
3.4	Démonstration expérimentale : Autoencodeur Topologique sur un anneau 2D	10
3.4.1	Protocole expérimental	10
3.4.2	Analyse comparative des résultats	10
4	Expérimentations Avancées : Benchmarks et Architectures	11
4.1	Limites des approches classiques et refonte du protocole	11
4.1.1	Génération d'un jeu de données "extrême"	11
4.1.2	Les 5 pipelines en compétition	12
4.2	Analyse des résultats et performances	13
4.2.1	Effondrement des approches classiques et limites de l'expertise	13
4.2.2	Efficacité des vectorisations statiques en régime "Low Data"	13
4.2.3	Apprentissage de bout en bout : puissance et exigence de la donnée	14
4.2.4	Validation statistique par validation croisée stratifiée	14
4.3	Implémentation d'une couche PersLay "from scratch" en PyTorch	16
5	Conclusion et Perspectives	16
A	Code source : Implémentation du réseau topologique	18

Table des figures

1	Illustration du Théorème du Nerf : équivalence entre le recouvrement continu (boules bleues) et l'objet combinatoire (nerf rouge) (nos travaux précédents). . .	4
2	Impact du rayon sur la reconstruction d'un cercle : poussière de points (gauche), topologie correcte (centre), bouchage du trou (droite) (nos travaux précédents). .	4
3	Analyse des cercles concentriques : nuage de points, diagramme et code-barres de persistance. Les deux points détachés en H_1 identifient les deux cercles malgré le bruit (nos travaux précédents).	5
4	Matrices de confusion : échec de la baseline géométrique (gauche) face à la réussite de l'approche TDA (droite) (nos travaux précédents).	6
5	Signature moyenne des classes : λ_2 ne s'active que pour le "8", offrant un critère de décision robuste (nos travaux précédents).	6
6	Comparaison des espaces latents et des reconstructions entre un autoencodeur classique (haut) et un autoencodeur régularisé topologiquement (bas) sur des données en anneau. Les courbes de loss sont présentées à gauche.	10
7	Échantillons du dataset complexe. De gauche à droite : un amas, un anneau et deux anneaux. Les dégradations isométriques et l'ajout massif de bruit de fond uniforme rendent la structure globale indiscernable pour une approche spatiale classique.	12
8	Comparaison des précisions (<i>Accuracy</i>) sur le dataset complexe. On observe la supériorité des approches topologiques sur la baseline géométrique.	13
9	Temps de calcul par pipeline. On note le coût supérieur des approches <i>Deep Learning</i> dû à la phase d'entraînement, face à la rapidité d'inférence des modèles SVM.	14
10	Distribution des accuracies par pipeline sur 10 folds de validation croisée stratifiée. Les losanges rouges indiquent la moyenne, les traits oranges la médiane. La ligne pointillée marque le seuil du hasard (33.3%).	15

1 Introduction

1.1 Contexte et motivations

L'analyse de données contemporaine fait face à une explosion de la quantité et de la complexité des informations disponibles, rendant centrale l'identification de structures sous-jacentes pour l'apprentissage statistique. Bien que l'apprentissage géométrique ait permis de modéliser avec succès des données structurées dans des espaces non euclidiens [8], ces méthodologies basées sur des voisinages locaux peinent souvent à capturer des invariants macro-structuraux globaux. En particulier, les propriétés topologiques comme les cycles de longue portée ou les cavités multidimensionnelles échappent fréquemment aux approches classiques [8]. Dans ce contexte, l'Analyse Topologique des Données (TDA) propose des outils mathématiques robustes pour inférer ces caractéristiques à partir de données souvent bruitées. La fusion de cette discipline avec les réseaux de neurones a donné naissance au *Topological Deep Learning* (TDL), une approche visant à intégrer la topologie comme un composant de l'optimisation [8].

1.2 Objectifs du projet et bilan des travaux précédents

L'objectif fondamental de ce projet est d'exploiter les invariants topologiques pour renforcer les capacités de classification et de représentation des modèles d'apprentissage. Dans nos travaux précédents, nous avons réalisé une première phase d'appropriation théorique et de prise en main technique de la TDA. Cette étape a permis de comprendre en profondeur les concepts d'homologie persistante et de maîtriser les outils numériques de la librairie Gudhi.

Cette phase initiale s'est conclue par une preuve de concept réussie dans le domaine de la classification de formes géométriques 2D. En utilisant des descripteurs tels que les paysages de persistance (*Persistence Landscapes*) [2], nous avons démontré, dans nos travaux précédents, que l'approche topologique surpassait les méthodes géométriques classiques face à des transformations isométriques et au bruit. Ces résultats ont confirmé la robustesse et la stabilité des signatures topologiques pour distinguer des structures complexes, comme des cercles simples de figures en huit.

1.3 Évolution de la problématique : vers le Topological Deep Learning

Cependant, l'approche adoptée dans nos travaux précédents présentait une limite majeure : l'extraction des caractéristiques topologiques restait « statique » et déconnectée de la tâche d'apprentissage finale. Les paramètres de vectorisation étaient fixés *a priori*, empêchant la représentation topologique de s'adapter dynamiquement aux spécificités des données durant l'entraînement.

La problématique de ce rapport final évolue donc vers une intégration dynamique de la topologie au sein même des architectures de Deep Learning. L'enjeu est de transformer les descripteurs statiques en signatures paramétrées entraînaibles de bout en bout (*end-to-end*) [6, 3]. Cette démarche nécessite de surmonter des défis mathématiques importants, notamment liés à la nature non différentiable des calculs d'homologie, afin d'autoriser la rétropropagation du gradient à travers les transformations topologiques [3].

1.4 Structure du rapport

Le présent rapport suit une progression logique. La section 2 offre une synthèse des fondations théoriques, du nuage de points aux complexes simpliciaux. La section 3 détaille l'intégration de la topologie dans le Deep Learning via des architectures différentiables (PersLay) et des mécanismes de régularisation [3, 7]. La section 4 présente un benchmark comparatif sur des données complexes ainsi que l'implémentation de notre propre couche topologique. Enfin, la section 5

vient clore ce rapport en dressant le bilan de nos travaux et en ouvrant sur les perspectives d'avenir du domaine.

2 Fondations Théoriques : De la Géométrie à la Topologie

Cette section synthétise les fondements théoriques et les résultats expérimentaux établis lors de nos travaux précédents, qui constituent le socle du présent rapport.

2.1 Des complexes simpliciaux à l'homologie persistante

L'enjeu central de la TDA est de jeter un pont entre le monde discret des nuages de points et le monde continu de la topologie [8]. Pour cela, on construit un *complexe simplicial* sur le nuage de points à une échelle ϵ donnée. En pratique, le complexe de Vietoris-Rips, qui crée un k -simplexe dès que $k + 1$ points sont deux à deux à distance inférieure à 2ϵ , est privilégié pour son efficacité calculatoire. La validité de cette discrétisation repose sur le *Théorème du Nerf*, garantissant l'équivalence homotopique entre le complexe et l'union des boules de recouvrement (Figure 1).

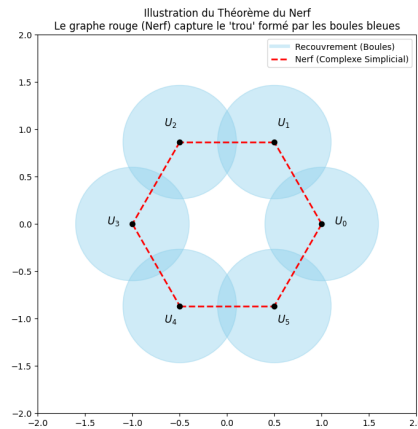


FIGURE 1 – Illustration du Théorème du Nerf : équivalence entre le recouvrement continu (boules bleues) et l'objet combinatoire (nerf rouge) (nos travaux précédents).

Le choix du rayon ϵ est critique : trop petit, les points restent déconnectés ; trop grand, les cavités sont bouchées (Figure 2). La *filtration* résout ce dilemme en faisant varier ϵ de 0 à l'infini, engendrant une séquence emboîtée $\emptyset \subseteq K_{\epsilon_0} \subseteq K_{\epsilon_1} \subseteq \dots$.

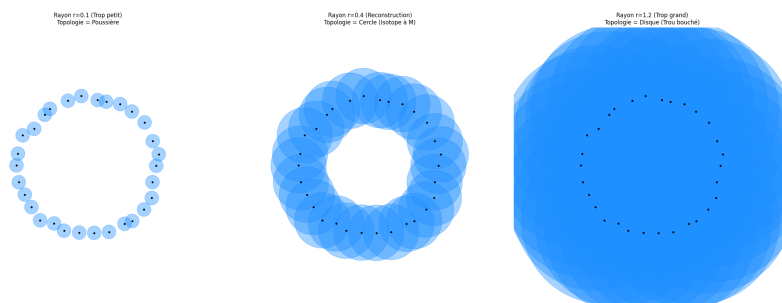


FIGURE 2 – Impact du rayon sur la reconstruction d'un cercle : poussière de points (gauche), topologie correcte (centre), bouchage du trou (droite) (nos travaux précédents).

L'*homologie persistante* observe alors l'évolution topologique à travers toutes les échelles simultanément [8]. Chaque caractéristique est associée à une paire (b, d) de naissance/mort ; sa

persistance $d - b$ distingue le signal robuste (forte persistance) du bruit topologique (faible persistance, proche de la diagonale) [5]. Les nombres de Betti β_k résument cette information : β_0 compte les composantes connexes, β_1 les cycles, et β_2 les cavités. Le **diagramme de persistance** et le **code-barres** en sont les représentations visuelles canoniques (Figure 3).

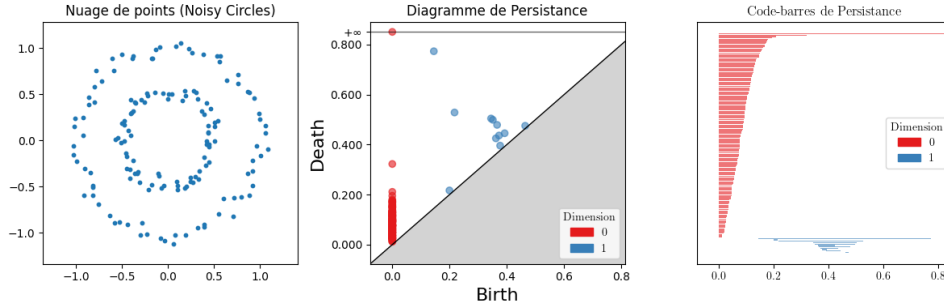


FIGURE 3 – Analyse des cercles concentriques : nuage de points, diagramme et code-barres de persistance. Les deux points détachés en H_1 identifient les deux cercles malgré le bruit (nos travaux précédents).

2.2 Stabilité théorique : distance de Bottleneck

L'applicabilité de la TDA aux données réelles repose sur le *théorème de stabilité* [5]. La distance de Bottleneck d_B entre deux diagrammes X et Y est définie par :

$$d_B(X, Y) = \inf_{\gamma} \sup_{x \in X} \|x - \gamma(x)\|_{\infty}$$

Le théorème établit que pour des fonctions continues f et g :

$$d_B(D(f), D(g)) \leq \|f - g\|_{\infty}$$

Cette inégalité prouve que l'application associant un nuage de points à son diagramme est 1-Lipschitzienne : une petite perturbation des données n'engendre qu'une petite perturbation du diagramme [5].

2.3 Bilan de la preuve de concept et limites de l'approche statique

Nos travaux précédents ont validé l'intérêt pratique de la TDA par une classification de formes 2D : discriminer un cercle ($\beta_1 = 1$) d'une figure en "8" ($\beta_1 = 2$), sous translations, rotations aléatoires et bruit gaussien. Un *Random Forest* sur coordonnées brutes n'atteint que 56,7% de précision, tandis que l'approche TDA par *Persistence Landscapes* [2] atteint 100% (Figure 4). Le critère discriminant appris est l'activation de la couche λ_2 du landscape, signature du second cycle du "8" (Figure 5).

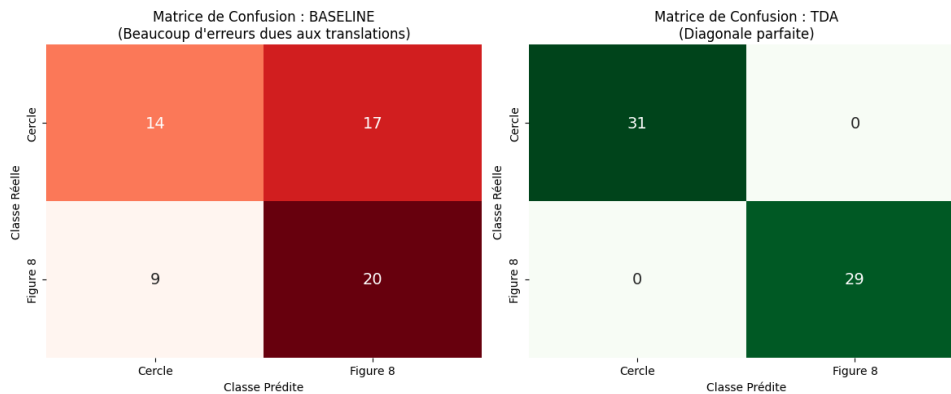


FIGURE 4 – Matrices de confusion : échec de la baseline géométrique (gauche) face à la réussite de l'approche TDA (droite) (nos travaux précédents).

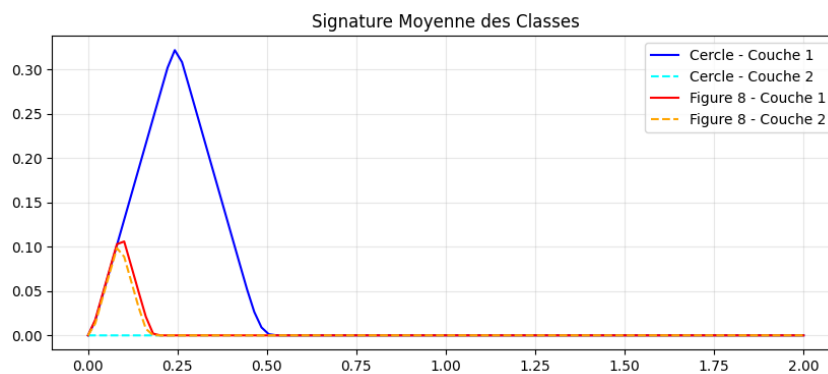


FIGURE 5 – Signature moyenne des classes : λ_2 ne s'active que pour le "8", offrant un critère de décision robuste (nos travaux précédents).

Cependant, cette approche "statique" en deux étapes présente des limites structurelles. L'extraction des caractéristiques est fixée *a priori*, indépendamment de la tâche finale : la représentation topologique n'est pas optimisée pour minimiser l'erreur de classification [3]. De plus, le coût calculatoire de l'homologie persistante limite le passage à l'échelle. Ces constats imposent une transition vers le *Topological Deep Learning* [8, 6] : intégrer la topologie comme un composant dynamique et différentiable au sein des réseaux de neurones, rendant les signatures entraînaibles de bout en bout.

3 Intégration de la Topologie dans le Deep Learning

3.1 De l'extraction statique à l'apprentissage de bout en bout

Si les approches de vectorisation statique explorées précédemment, telles que les *Persistence Landscapes* et les *Persistence Images*, ont validé le potentiel de la topologie algébrique pour la classification [2, 1], elles souffrent d'une limitation structurelle inhérente aux modèles d'apprentissage "en deux étapes" [3].

3.1.1 Le problème de l'optimisation déconnectée

Dans un pipeline statique, l'extraction des caractéristiques topologiques est aveugle à la tâche d'apprentissage effectuée en aval [3]. Les hyperparamètres régissant cette vectorisation, tels que la variance du lissage gaussien pour une image de persistance ou la résolution de la

grille de discrétisation, sont fixés *a priori* par l'expert ou via une recherche exhaustive coûteuse (*Grid Search*) [3]. Par conséquent, la représentation topologique ne s'adapte pas pour minimiser dynamiquement la fonction de perte (*Loss*) du réseau de neurones [3].

3.1.2 Vers des signatures entraînables

L'évolution naturelle et la plus radicale du *Topological Deep Learning* a consisté à intégrer l'homologie persistante directement à l'intérieur du graphe de calcul du réseau de neurones [6]. L'objectif est de permettre la rétropropagation du gradient à travers les transformations topologiques, convertissant ainsi les descripteurs statiques en signatures paramétrées dont la représentation optimale est apprise de bout en bout [6].

Dans ces architectures, la couche d'entrée neuronale calcule une projection paramétrée des points de persistance [6]. Le réseau définit un ensemble d'éléments structurants (par exemple, des distributions de probabilités bivariées) dont les centres et les écarts-types sont initialisés aléatoirement, puis traités comme des poids du réseau optimisables par la descente de gradient [6]. Lors de l'entraînement, la rétropropagation permet à ces centres de se déplacer de manière autonome dans le plan de persistance pour "chercher" les régions où les structures topologiques sont les plus discriminantes [6].

3.1.3 Le défi de l'invariance par permutation

Toutefois, l'intégration directe de ces diagrammes soulève un obstacle mathématique majeur : un diagramme de persistance est, par définition, un multi-ensemble de points non ordonnés et invariant par permutation [3]. Un réseau de neurones standard, tel qu'un perceptron multicouche (MLP), ingérant directement les coordonnées de ces points serait fortement sensible à l'ordre dans lequel la liste lui est présentée [3].

Ce blocage impose de concevoir des architectures adaptées aux ensembles. La couche *PersLay*, détaillée ci-après, répond à ce besoin.

3.2 L'architecture PersLay : une couche neuronale topologique

Pour ingérer un diagramme de persistance, un réseau de neurones doit nécessairement gérer la nature non ordonnée de cet objet mathématique [3]. Un diagramme étant un multi-ensemble de points, une architecture classique serait incapable de garantir l'invariance par permutation requise.

3.2.1 Cadre théorique et formulation mathématique

L'architecture *PersLay* (*Persistence Layer*) résout cette problématique en s'appuyant sur les théorèmes de représentation des ensembles, notamment le cadre mathématique des *Deep Sets* [3, 9]. *PersLay* fournit une équation générique, différentiable et invariante par permutation, capable d'approximer n'importe quelle fonction symétrique opérant sur un diagramme de persistance Dg [3].

L'opération fondamentale d'une couche *PersLay* est formulée ainsi :

$$PersLay(Dg) := op(\{w(p) \cdot \phi(p)\}_{p \in Dg})$$

Pour fixer les idées, considérons un diagramme de persistance $Dg = \{p_1, p_2, p_3\}$ composé de trois points dans le plan naissance-mort, et une couche *PersLay* équipée de $q = 2$ éléments structurants. L'opération se décompose en trois étapes séquentielles : (i) chaque point p_i est projeté en un vecteur de dimension q par φ , (ii) ce vecteur est pondéré par un scalaire $w(p_i)$ reflétant l'importance du point, et (iii) l'ensemble des $|Dg|$ vecteurs pondérés est agrégé en un unique vecteur de sortie de dimension q par l'opérateur op , invariant par permutation. Le

résultat est un vecteur de taille fixe $\mathbb{R}^q$, indépendant de la cardinalité du diagramme d'entrée, directement exploitable par les couches denses situées en aval.

Cette architecture tire sa modularité de trois opérateurs distincts [3] :

- **La fonction de transformation** $\phi(p)$: Elle projette chaque point $p = (b, d)$ du diagramme dans un espace vectoriel. Il peut s'agir de fonctions "tentes" triangulaires ou de distributions gaussiennes [3].
- **La fonction de pondération** $w(p)$: Elle module l'influence de chaque point, permettant notamment d'atténuer le bruit topologique situé près de la diagonale. Ses paramètres internes (comme une grille de poids) sont **optimisés durant l'entraînement** par la descente de gradient [3].
- **L'opérateur d'agrégation** op : Cette fonction (telle que la somme globale ou le maximum global) opère sur l'ensemble transformé et garantit la stricte invariance par permutation de la sortie [3].

Exemple concret : gaussiennes bivariées. Dans notre implémentation (voir Annexe A et Pipeline E de la section 4), la transformation φ est réalisée par q gaussiennes bivariées paramétrées par leurs centres $t_j \in \mathbb{R}^2$ et leurs variances $\sigma_j^2 > 0$. Pour un point $p = (b, d)$ du diagramme, la j -ème composante du vecteur transformé s'écrit :

$$\varphi_j(p) = \exp\left(-\frac{\|p - t_j\|^2}{2\sigma_j^2}\right), \quad j = 1, \dots, q \quad (1)$$

Chaque gaussienne agit comme un *détecteur localisé* dans le plan de persistance : elle produit une activation forte pour les points du diagramme situés à proximité de son centre t_j , et une activation quasi nulle pour les points éloignés. Lors de l'initialisation, les centres sont placés aléatoirement dans le plan (b, d) . Au fil de l'entraînement, la rétropropagation du gradient déplace ces centres vers les régions du diagramme les plus informatives pour la tâche de classification. Concrètement, si la distinction entre deux classes repose sur la présence d'un cycle de forte persistance (un point éloigné de la diagonale), certaines gaussiennes migreront vers cette zone pour y concentrer leur pouvoir discriminant, tandis que d'autres pourront se positionner près de la diagonale pour quantifier le niveau de bruit topologique. Ce mécanisme d'attention spatiale apprise constitue l'avantage fondamental de PersLay sur les vectorisations statiques, où la position et la largeur des noyaux sont fixées *a priori* sans retour de la tâche supervisée.

3.2.2 Unification et différentiabilité

L'innovation principale de PersLay réside dans son pouvoir d'unification. En paramétrant spécifiquement ces trois opérateurs, la couche peut mathématiquement recréer la majorité des vectorisations statiques de la littérature [3]. Par exemple, associer une transformation gaussienne à un opérateur de sommation génère une *Persistence Image*, tandis qu'associer une transformation triangulaire à une maximisation mime un *Persistence Landscape* [3]. La différence fondamentale est que les centres, les variances et les poids ne sont plus figés, mais appris dynamiquement pour maximiser les performances sur la tâche visée [3].

De plus, des recherches mathématiques récentes ont prouvé que la fonction convertissant une filtration sur un complexe simplicial en un diagramme de persistance est génériquement différentiable (presque partout) [3]. Cela garantit la viabilité de l'algorithme de descente de sous-gradient stochastique sur l'ensemble de l'architecture [3].

3.2.3 Vers la régularisation de l'espace latent

Si l'architecture PersLay permet d'utiliser la topologie comme une variable d'entrée parfaitement optimisée, la flexibilité des graphes de calcul profond ouvre une seconde voie d'intégration. Au lieu d'injecter l'information topologique à l'entrée du réseau, il est possible de l'utiliser en

sortie, au niveau de la fonction de perte (*Loss*), pour contraindre la géométrie de l'espace latent. Ce mécanisme, appelé régularisation topologique, fait l'objet de la section suivante.

3.3 Régularisation Topologique : Préserver la Continuité de l'Espace Latent

L'apprentissage profond s'appuie massivement sur des architectures de réduction de dimension, telles que les Autoencodeurs (AE), pour projeter des données complexes dans un espace latent [8]. Cependant, les fonctions de perte de reconstruction classiques, comme l'Erreur Quadratique Moyenne (MSE), mesurent exclusivement des distances locales point par point.

3.3.1 Pathologies de l'espace latent

Cette focalisation purement locale induit des distorsions géométriques sévères [7]. Deux pathologies structurelles majeures affectent régulièrement l'espace latent :

- **Les déchirures (*tearing*)** : Des points topologiquement connexes dans l'espace d'entrée sont brutalement séparés dans l'espace latent, détruisant la continuité de la variété.
- **Les raccourcis (*short-circuiting*)** : Des régions géométriquement éloignées à l'origine se replient et se superposent, créant de fausses similarités.

Ces pathologies peuvent être formalisées en termes de distorsion des distances entre paires de points critiques. Soit $f_{\text{enc}} : \mathcal{X} \rightarrow \mathcal{Z}$ l'encodeur. On note $\mathcal{P}_X$ l'ensemble des *persistence pairings* dans l'espace d'entrée, c'est-à-dire les paires de points (x_i, x_j) dont la distance définit la naissance ou la mort d'une caractéristique topologique dans la filtration de Vietoris-Rips. Une **déchirure** se produit lorsque, pour une paire $(x_i, x_j) \in \mathcal{P}_X$ associée à un cycle persistant, la distance latente $\|f_{\text{enc}}(x_i) - f_{\text{enc}}(x_j)\|$ devient disproportionnellement grande par rapport à $\|x_i - x_j\|$, brisant la continuité locale. Inversement, un **raccourci** survient lorsque deux points x_i, x_j éloignés dans $\mathcal{X}$ vérifient $\|f_{\text{enc}}(x_i) - f_{\text{enc}}(x_j)\| \approx 0$, créant des identifications topologiques artificielles absentes des données d'origine. Dans les deux cas, le diagramme de persistance de l'espace latent $D(f_{\text{enc}}(\mathcal{X}))$ diffère qualitativement de celui de l'espace d'entrée $D(\mathcal{X})$.

3.3.2 La Topological Loss

Pour pallier ces distorsions, Moor et al. [7] proposent un terme de régularisation fondé sur les *persistence pairings*. L'idée clé est la suivante : chaque caractéristique topologique détectée dans la filtration de Vietoris-Rips est définie par une paire d'arêtes critique — l'une provoquant sa naissance, l'autre sa mort. Ces arêtes identifient des paires de points (x_i, x_j) dans le nuage d'entrée dont la distance mutuelle encode directement l'information topologique.

La perte topologique impose alors que ces distances soient préservées proportionnellement dans l'espace latent. Formellement, soit $\mathcal{P}_X^{(k)}$ l'ensemble des paires de persistance de dimension k calculées sur l'espace d'entrée $\mathcal{X}$, et $\mathcal{P}_Z^{(k)}$ l'ensemble analogue sur l'espace latent $\mathcal{Z} = f_{\text{enc}}(\mathcal{X})$. La perte proposée dans [7] s'écrit :

$$\mathcal{L}_{\text{topo}} = \frac{1}{|\mathcal{P}_X^{(k)}|} \sum_{(i,j) \in \mathcal{P}_X^{(k)}} (\|x_i - x_j\| - \|z_i - z_j\|)^2 + \frac{1}{|\mathcal{P}_Z^{(k)}|} \sum_{(i,j) \in \mathcal{P}_Z^{(k)}} (\|x_i - x_j\| - \|z_i - z_j\|)^2 \quad (2)$$

où $z_i = f_{\text{enc}}(x_i)$ désigne la représentation latente du point x_i .

Le premier terme pénalise les déchirures : si une paire (x_i, x_j) définit un cycle persistant dans l'entrée, sa distance doit être conservée dans l'espace latent. Le second terme pénalise les raccourcis : si une paire (z_i, z_j) crée un cycle *uniquement* dans l'espace latent (absent de l'entrée), la perte force la distance latente à se conformer à la distance originale, dissipant ainsi les fausses identifications. La fonction de perte totale combine alors la reconstruction et la régularisation :

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \mathcal{L}_{\text{topo}} \quad (3)$$

où $\lambda > 0$ contrôle le compromis entre fidélité de reconstruction et préservation topologique.

3.4 Démonstration expérimentale : Autoencodeur Topologique sur un anneau 2D

Afin de valider concrètement l'impact de cette régularisation, nous avons implémenté et évalué un Autoencodeur Topologique (TopoAE).

3.4.1 Protocole expérimental

L'expérience repose sur un jeu de données 2D synthétique en forme de couronne (anneau), généré avec un rayon intérieur de 0,7 et un rayon extérieur de 1,3. Cette structure présente une topologie non triviale caractérisée par un cycle principal H_1 (le trou central).

L'architecture de l'autoencodeur est un Perceptron Multicouche (MLP) compact : un encodeur compressant les données 2D vers un espace latent 2D via deux couches cachées de 32 neurones (activation ReLU), suivi d'un décodeur symétrique. Le terme de régularisation topologique a été calculé via un complexe de Vietoris-Rips sur un sous-échantillonnage de 80 points. La fonction de perte totale s'exprime ainsi : $\mathcal{L} = \mathcal{L}_{MSE} + \lambda \mathcal{L}_{topo}$ (avec $\lambda = 0.5$), évaluée toutes les 5 époques pour limiter le coût computationnel.

Notre implémentation adopte une version simplifiée de la perte de l'équation (2), s'inspirant davantage de l'approche par nombres de Betti cibles proposée par Clough et al. [4] dans le contexte de la segmentation d'images. Le terme $\mathcal{L}_{topo}$ est estimé sur un sous-échantillonnage de 80 points via un complexe de Vietoris-Rips et combine deux pénalités : (i) l'écart quadratique entre le nombre de cycles H_1 significatifs détectés dans l'espace latent et le nombre attendu, et (ii) un terme favorisant une forte persistance du cycle principal. Contrairement au cadre de Moor et al. [7], cette perte n'opère pas sur les distances entre paires de points critiques. Contrairement à Clough et al., elle n'est pas différentiable à travers le calcul de persistance (les coordonnées latentes sont détachées du graphe de calcul). Elle constitue néanmoins une heuristique suffisante pour notre démonstration sur des données 2D, où la structure topologique à préserver se réduit à un unique cycle.

3.4.2 Analyse comparative des résultats

Les modèles ont été entraînés sur 200 époques avec l'optimiseur Adam. La Figure 6 illustre la différence marquée de comportement entre les deux approches.

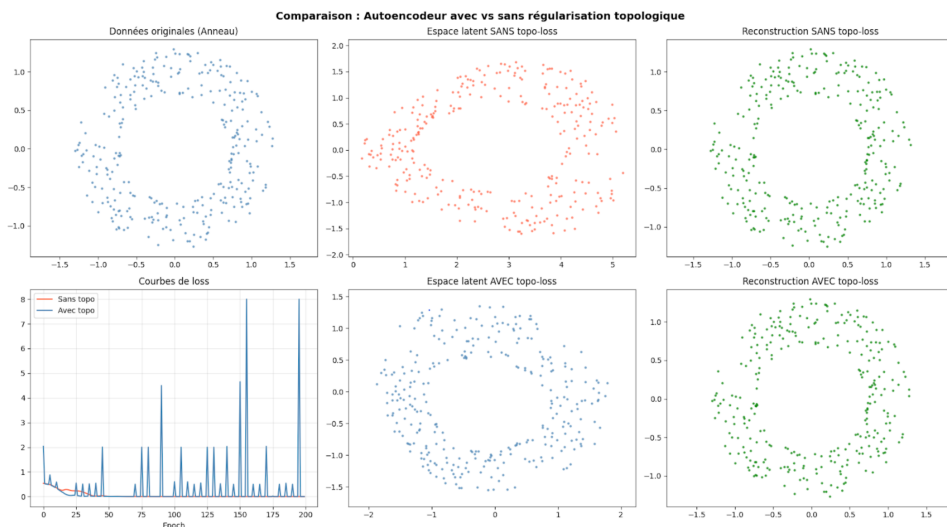


FIGURE 6 – Comparaison des espaces latents et des reconstructions entre un autoencodeur classique (haut) et un autoencodeur régularisé topologiquement (bas) sur des données en anneau. Les courbes de loss sont présentées à gauche.

L'autoencodeur standard (sans régularisation topologique) tend systématiquement à écraser l'anneau dans l'espace latent. Le modèle privilégie la minimisation absolue de l'erreur de reconstruction (MSE) au détriment de la structure, faisant disparaître le trou central au profit d'une distribution compacte.

À l'inverse, l'autoencodeur régularisé (avec $\mathcal{L}_{topo}$) contraint l'espace latent à conserver l'intégrité du cycle H_1 de l'anneau. L'analyse topologique des espaces latents confirme la présence d'un cycle significatif conforme à la forme d'origine. Ce résultat est obtenu au prix d'un léger compromis (*trade-off*) : une erreur de reconstruction (MSE) marginalement plus élevée.

Cette expérimentation prouve que la topologie peut façonner activement les représentations internes d'un réseau. Cette approche s'avère importante pour les tâches génératives ou l'apprentissage de variétés, où la préservation de la structure globale dans l'espace latent est souvent plus importante qu'une optimisation pixel par pixel [7].

4 Expérimentations Avancées : Benchmarks et Architectures

4.1 Limites des approches classiques et refonte du protocole

Lors de nos expérimentations précédentes, nous avons constaté que l'ensemble des méthodes topologiques atteignait des performances parfaites (100% de précision) sur notre jeu de données initial [8]. Ce phénomène de "plafond de verre" s'explique par la trivialité topologique des données utilisées : le signal géométrique (le nombre de trous) était si pur que n'importe quelle heuristique de base suffisait à le capturer de manière déterministe.

Cependant, pour comparer rigoureusement l'expressivité d'une architecture neuronale topologique face à des méthodes statiques, il est impératif de concevoir un problème complexe où la géométrie classique échoue et où l'expertise humaine est mise en échec par un bruit de fond important.

4.1.1 Génération d'un jeu de données "extrême"

Nous avons donc redéfini notre protocole expérimental pour générer un jeu de données de 240 nuages de points (80 par classe) répartis en trois catégories : des amas gaussiens sans structure ($\beta_1 = 0$), des anneaux simples de rayon variable ($\beta_1 = 1$), et des paires d'anneaux disjoints ($\beta_1 = 2$).

Pour détruire l'information spatiale et éprouver la robustesse des modèles, chaque nuage de points a subi de sévères dégradations :

- **Isométries aléatoires** : Application de matrices de rotation ($\theta \in [0, 2\pi]$) et de vecteurs de translation ($t \in [-3, 3]^2$) pour rendre les coordonnées brutes absolues inexploitable.
- **Variabilité d'échelle** : Les rayons des anneaux sont tirés aléatoirement, générant des cycles de tailles et de persistances très disparates.
- **Bruit de fond extrême** : Outre un bruit gaussien local, l'ajout massif de 100 points parasites uniformément répartis dans l'espace $[-5, 5]^2$ noie visuellement les structures principales.

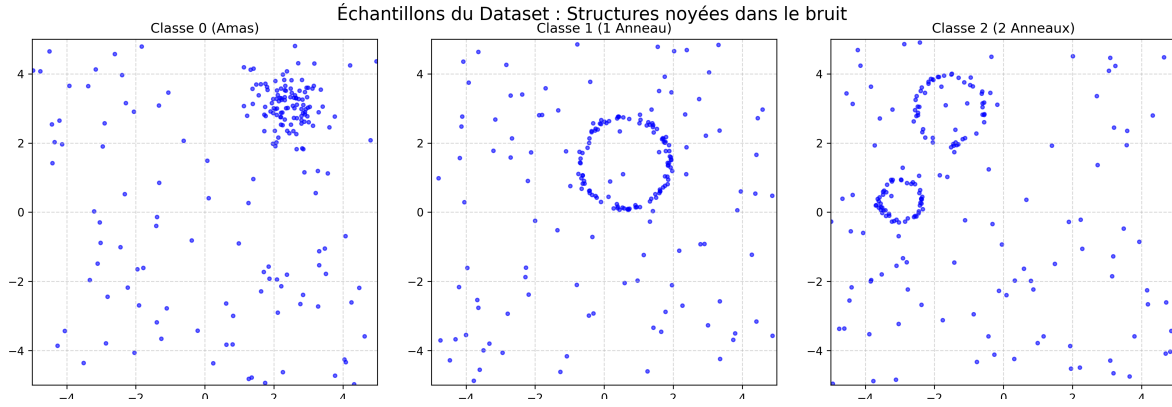


FIGURE 7 – Échantillons du dataset complexe. De gauche à droite : un amas, un anneau et deux anneaux. Les dégradations isométriques et l’ajout massif de bruit de fond uniforme rendent la structure globale indiscernable pour une approche spatiale classique.

4.1.2 Les 5 pipelines en compétition

Afin d’isoler l’apport spécifique du *Topological Deep Learning*, nous avons mis en compétition cinq architectures distinctes sur ce même jeu de données :

- **Pipeline 0 (Baseline DeepSets)** : Un réseau de neurones géométrique classique agissant comme témoin. Il ingère les coordonnées brutes sans information topologique, applique un Perceptron Multicouche (MLP) point par point, puis un *Max Pooling* symétrique pour garantir l’invariance par permutation.
- **Pipeline A (Features manuelles + Random Forest)** : L’approche experte rigide. Elle utilise des descripteurs précalculés sur les diagrammes de persistance H_1 (persistance maximale, somme des persistances, et comptage conditionnel des cycles).
- **Pipelines B et C (Vectorisations Statiques + SVM)** : L’utilisation de représentations continues et paramétrées *a priori*, à savoir les *Persistence Landscapes* [2] et les *Betti Curves*, couplées à un classifieur à vaste marge (SVM).
- **Pipeline D (Pseudo-PersLay + MLP)** : Une approximation statique simulant une couche topologique neuronale. Le diagramme est projeté sur une grille 1D de fonctions gaussiennes fixes avant de nourrir un MLP.
- **Pipeline E (True PersLay End-to-End)** : L’implémentation complète de la couche neuronale *PersLay* [3]. Les centres et les variances des gaussiennes bivariées sont définis comme des poids du graphe de calcul et optimisés dynamiquement par la descente de gradient durant l’entraînement [3].

L’ensemble des pipelines partage un socle d’hyperparamètres communs, fixés selon des heuristiques de la littérature ou des contraintes computationnelles, et maintenus constants à travers toutes les expériences pour garantir l’équité de la comparaison.

Le nombre de gaussiennes bivariées ($q = 60$) utilisé dans les Pipelines D et E est choisi conformément à l’ordre de grandeur recommandé par Carrière et al. [3], qui utilisent $q \in [20, 100]$ selon la complexité du jeu de données. La valeur $q = 60$ offre un compromis entre expressivité et coût mémoire pour des diagrammes dont la cardinalité typique observée est de l’ordre de 30 à 80 points en H_1 .

Le sous-échantillonnage à 80 points pour le calcul du complexe de Rips dans la perte topologique est dicté par la complexité cubique de l’algorithme de persistance, qui rend le calcul sur l’intégralité des 200 points prohibitif à chaque époque d’entraînement.

La résolution des vectorisations statiques (Landscapes et Betti Curves, $r = 80$) correspond à la valeur par défaut de la librairie `gudhi.representations`. Le nombre d’époques d’entraînement (150) et le taux d’apprentissage ($\eta = 0.01$) des réseaux neuronaux (Pipelines 0, D et E) sont des valeurs standards pour des architectures MLP de petite taille sur des jeux de données

réduits.

Ces choix n'ont pas fait l'objet d'une optimisation systématique (par exemple via une recherche sur grille avec validation croisée imbriquée). Cette limitation est cohérente avec la taille réduite du jeu de données ($n = 240$), qui rend une telle procédure sujette à une forte variance d'estimation. Les résultats présentés doivent donc être interprétés comme une comparaison à *hyperparamètres raisonnables fixés*, et non comme une borne supérieure des performances atteignables par chaque pipeline.

4.2 Analyse des résultats et performances

Les résultats obtenus lors du benchmark (synthétisés dans la Figure 8) valident l'intérêt des approches topologiques tout en révélant des nuances importantes sur le choix des modèles en fonction du volume de données disponible.

4.2.1 Effondrement des approches classiques et limites de l'expertise

Le premier constat majeur est l'échec critique de la *Baseline* géométrique (Pipeline 0), qui plafonne à une précision de 36,11%. Ce score, proche du hasard pur (33,3% pour trois classes), confirme que les réseaux neuronaux spatiaux standards sont incapables de reconstruire la structure des formes lorsque celles-ci sont noyées dans un bruit de fond uniforme et soumises à des isométries.

Parallèlement, l'approche par caractéristiques manuelles (Pipeline A) sous-performe également avec 43,06% de précision. Cet échec souligne les limites de l'expertise humaine : le bruit de fond génère une multitude de micro-cycles éphémères qui "polluent" le diagramme de persistance. Un seuil de filtration fixe, tel que celui utilisé dans nos travaux précédents, devient inopérant car le bruit topologique se confond désormais avec le signal des structures de petite échelle.

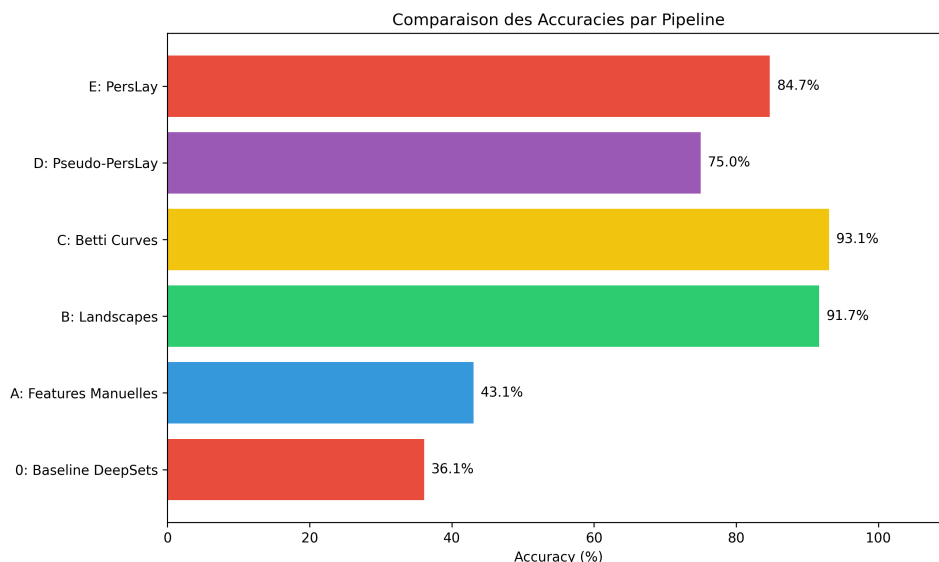


FIGURE 8 – Comparaison des précisions (*Accuracy*) sur le dataset complexe. On observe la supériorité des approches topologiques sur la baseline géométrique.

4.2.2 Efficacité des vectorisations statiques en régime "Low Data"

À l'inverse, les vectorisations statiques couplées à des classifieurs à vaste marge (SVM) affichent les meilleures performances, avec 91,67% pour les *Landscapes* et 93,06% pour les courbes de Betti.

Cette réussite s'explique par la nature du classifieur utilisé. Les SVM sont théoriquement optimisés pour fonctionner en régime de faibles données (*Low Data*) grâce à la maximisation de la marge de séparation. Sur un échantillon réduit de 240 individus, ces modèles parviennent à extraire une règle de décision robuste à partir des vecteurs topologiques exhaustifs sans souffrir du surapprentissage (*overfitting*) qui guette les architectures plus profondes.

4.2.3 Apprentissage de bout en bout : puissance et exigence de la donnée

L'analyse de l'intégration au *Deep Learning* révèle l'apport incrémental du caractère entraînable des signatures. La comparaison entre le *Pseudo-PersLay* (75,00%) et le *True PersLay* (84,72%) met en évidence un gain de près de 10%. Ce bond de performance démontre l'efficacité de l'optimisation dynamique : en traitant les centres et variances des gaussiennes comme des paramètres entraînaibles, le réseau apprend de lui-même à ignorer les zones du diagramme saturées par le bruit pour se focaliser sur les régions discriminantes [3].

Toutefois, le fait que *PersLay* ne surpasse pas encore le SVM dans cette expérience s'explique par la "faim de données" (*data-hunger*) intrinsèque aux réseaux de neurones. L'optimisation simultanée d'une couche d'extraction topologique et d'un classifieur MLP nécessite un volume d'échantillons plus conséquent pour converger vers une solution globale supérieure aux méthodes à noyaux classiques [8]. La section suivante montrera que cette observation est en partie un artefact du découpage unique des données.

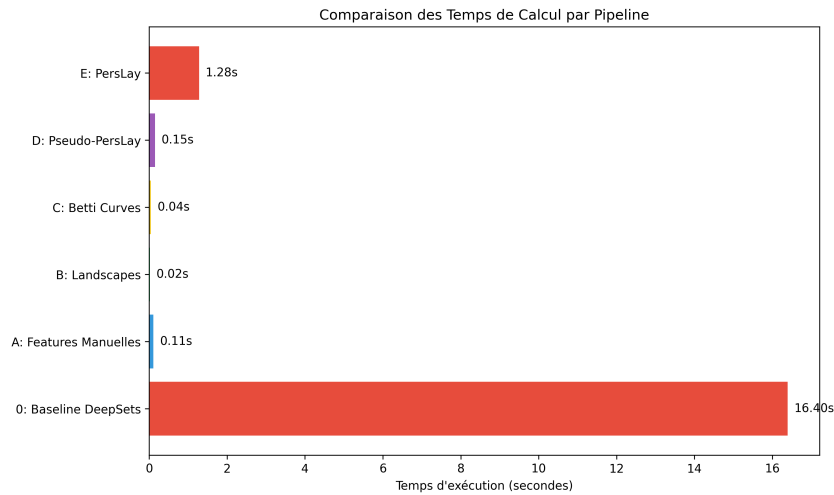


FIGURE 9 – Temps de calcul par pipeline. On note le coût supérieur des approches *Deep Learning* dû à la phase d'entraînement, face à la rapidité d'inférence des modèles SVM.

Enfin, l'analyse temporelle (Figure 9) montre un coût computationnel nettement plus élevé pour le *Pipeline E* (1,28s) par rapport aux méthodes statiques ($< 0,05s$). Ce compromis entre temps d'entraînement et adaptabilité est un facteur clé à considérer lors du passage à l'échelle industrielle.

En conclusion, ce benchmark démontre que si les vectorisations statiques restent d'excellents compromis pour des jeux de données restreints, le *Topological Deep Learning* offre une flexibilité supérieure pour apprendre des représentations *ad hoc* dans des environnements fortement bruités.

4.2.4 Validation statistique par validation croisée stratifiée

Les résultats présentés dans les sections précédentes reposaient sur un unique découpage train/test (70/30), ce qui, sur un échantillon de 240 individus, expose les conclusions à une forte

variance d'estimation. Afin de consolider statistiquement ces observations, nous avons conduit une validation croisée stratifiée à 10 *folds*, garantissant le respect des proportions de classes dans chaque partition. Les résultats complets sont synthétisés dans le Tableau 1 et la Figure 10.

TABLE 1 – Performances des pipelines en validation croisée stratifiée (10 folds). L'accuracy est exprimée en pourcentage.

Pipeline	Moyenne	$\pm$ Écart-type	Min	Max
0 : Baseline DeepSets	33.33	0.00	33.33	33.33
A : Features manuelles	40.00	7.73	25.00	50.00
B : Landscapes + SVM	90.83	5.83	83.33	100.00
C : Betti Curves + SVM	90.83	4.49	83.33	100.00
D : Pseudo-PersLay + MLP	70.00	6.92	58.33	83.33
E : True PersLay (E2E)	90.42	5.29	79.17	100.00

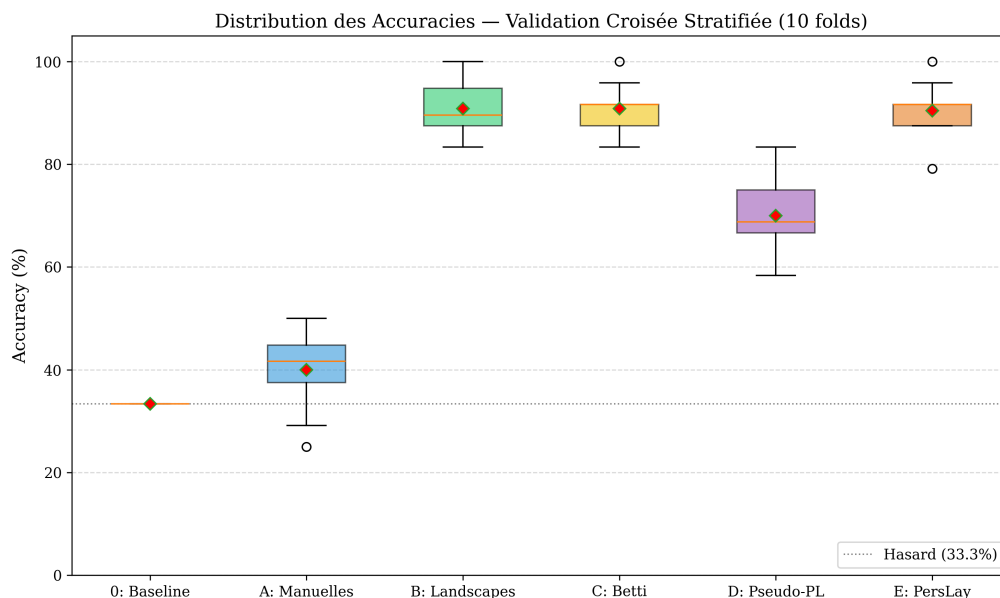


FIGURE 10 – Distribution des accuracies par pipeline sur 10 folds de validation croisée stratifiée. Les losanges rouges indiquent la moyenne, les traits oranges la médiane. La ligne pointillée marque le seuil du hasard (33.3%).

La validation croisée confirme et nuance les observations du split unique. Le Pipeline 0 (Baseline) s'effondre systématiquement à 33.33% avec un écart-type nul, confirmant l'incapacité structurelle — et non accidentelle — des réseaux spatiaux face aux isométries et au bruit de fond. Le Pipeline A (features manuelles) reste proche du hasard ($40.00 \pm 7.73\%$), validant l'insuffisance des heuristiques expertes sur ce régime de bruit.

Le résultat le plus notable concerne le Pipeline E (True PersLay). Sur le split unique initial, PersLay atteignait 84.72%, nettement en retrait par rapport aux SVM ($\sim 93\%$). La validation croisée révèle que cette sous-performance était en grande partie un artefact du découpage : en moyenne sur 10 folds, PersLay atteint $90.42 \pm 5.29\%$, un score statistiquement comparable aux Landscapes ($90.83 \pm 5.83\%$) et aux Betti Curves ($90.83 \pm 4.49\%$). Ce rattrapage suggère que l'architecture end-to-end, bien que plus sensible à la partition d'entraînement (variance légèrement supérieure), possède une capacité d'apprentissage équivalente aux méthodes statiques sur ce jeu de données.

La validation croisée fait émerger trois paliers de performance clairement séparés :

1. **Palier supérieur** ($\sim 90\%$) : Pipelines B, C et E — les trois approches exploitant des vectorisations topologiques complètes (statiques ou entraînables) atteignent des performances indiscernables.
2. **Palier intermédiaire** ($\sim 70\%$) : Pipeline D (Pseudo-PersLay) — la projection gaussienne 1D fixe perd une partie significative de l'information bidimensionnelle du diagramme.
3. **Palier inférieur** ($\leq 40\%$) : Pipelines 0 et A — les approches ne disposant pas d'une représentation topologique riche échouent sur ce régime de bruit.

L'écart entre le Pseudo-PersLay (D, 70.00%) et le True PersLay (E, 90.42%) est désormais quantifié de manière robuste à environ 20 points de pourcentage, renforçant la conclusion que l'optimisation dynamique des centres gaussiens constitue un apport déterminant de l'approche end-to-end.

4.3 Implémentation d'une couche PersLay "from scratch" en PyTorch

Afin de d'illustrer concrètement le fonctionnement du *Topological Deep Learning* et de justifier les performances obtenues lors de notre benchmark, nous avons implémenté notre propre couche différentiable, baptisée **TrueGaussianPersLay**, en utilisant le framework PyTorch. Le code source complet de cette implémentation, ainsi que son intégration dans un réseau de classification complet, est mis à disposition du lecteur en **Annexe A**.

L'objectif de cette implémentation est de transformer dynamiquement des diagrammes de persistance de taille variable en vecteurs d'entrée de taille fixe pour un réseau de neurones standard. Conformément à la théorie [3], notre classe repose sur des paramètres optimisables.

- **Initialisation des poids entraînaables** : Nous définissons un nombre fixé de gaussiennes bivariées (ici $q = 60$). Leurs centres (`self.centers`) sont initialisés aléatoirement dans le plan 2D, et le logarithme de leurs variances (`self.log_variances`) est initialisé à zéro. Ces tenseurs sont déclarés comme des `nn.Parameter`, ce qui les intègre au graphe de calcul pour être mis à jour par rétropropagation.
- **La transformation $\phi(p)$** : Lors de la passe *forward*, le code calcule la distance au carré entre chaque point p du diagramme en entrée et les centres des gaussiennes. L'application de la fonction exponentielle génère la transformation bivariée.
- **La pondération $w(p)$** : Pour moduler l'importance des points topologiques, nous utilisons directement la durée de vie du point (la différence entre sa mort et sa naissance) comme un poids naturel d'attention.
- **L'opérateur d'agrégation op et la gestion du padding** : Les diagrammes n'ayant pas tous le même nombre de points, les tenseurs sont "paddés" avec des zéros et un masque binaire (`masks`) est appliqué pour ignorer les points artificiels. L'agrégation finale utilise une somme globale (`torch.sum` sur la dimension des points), garantissant l'invariance par permutation.

Cette couche sur-mesure a ensuite été intégrée dans une architecture globale (**FullTopologicalNetwork**) en la chaînant à un perceptron multicouche (MLP) classique, prouvant ainsi la faisabilité technique de l'apprentissage topologique de bout en bout.

5 Conclusion et Perspectives

Ce rapport a présenté une exploration progressive de l'intégration de la topologie algébrique au sein des architectures d'apprentissage profond, depuis les fondements théoriques de l'homologie persistante jusqu'à l'implémentation et l'évaluation d'une couche neuronale topologique différentiable.

La première contribution de ce travail a été de démontrer, à travers un benchmark rigoureux sur des données synthétiques fortement dégradées, que les invariants topologiques constituent

un levier de classification puissant là où la géométrie euclidienne échoue. La validation croisée stratifiée à 10 folds a permis de quantifier cette supériorité de manière robuste : les trois approches exploitant des vectorisations topologiques complètes (Persistence Landscapes, Betti Curves et PersLay) atteignent un palier commun de $\sim 90\%$ de précision, contre $\sim 33\%$ pour la baseline spatiale.

La seconde contribution réside dans la mise en évidence du rôle de l’optimisation dynamique. L’écart de 20 points de pourcentage entre le Pseudo-PersLay (gaussiennes fixes) et le True PersLay (gaussiennes entraînaables) démontre concrètement l’apport du caractère *end-to-end* : le réseau apprend de lui-même à positionner ses détecteurs dans les régions discriminantes du plan de persistance, sans intervention experte. Toutefois, la validation croisée révèle également que, sur un jeu de données de taille réduite ($n = 240$), cet avantage ne suffit pas à surpasser les méthodes statiques couplées à des SVM, confirmant la sensibilité des architectures neuronales au volume de données disponible.

Enfin, l’expérimentation sur l’autoencodeur topologique a illustré une seconde modalité d’intégration : la topologie comme contrainte de régularisation. En pénalisant les distorsions de l’espace latent via une perte topologique inspirée de [4] et [7], nous avons montré qu’il est possible de préserver la structure globale des données au prix d’un compromis maîtrisé avec l’erreur de reconstruction.

Ces travaux comportent des limitations qu’il convient de reconnaître. L’ensemble des expérimentations repose sur des données synthétiques 2D, et le passage à des jeux de données réels de grande dimension reste à valider. L’implémentation de la perte topologique dans l’autoencodeur utilise une approximation simplifiée par rapport au cadre théorique de Moor et al. [7], dont l’impact sur des géométries plus complexes n’a pas été évalué. De plus, les hyperparamètres n’ont pas fait l’objet d’une optimisation systématique, et les résultats doivent être interprétés comme une comparaison à paramètres raisonnables fixés.

Plusieurs perspectives se dessinent naturellement. À court terme, l’augmentation du volume de données permettrait de tester l’hypothèse selon laquelle PersLay surpasserait les méthodes statiques en régime de données abondantes, comme le suggère la littérature [3]. L’exploration de filtrations alternatives, telles que les complexes d’Alpha, offrirait une meilleure efficacité en dimension supérieure à deux. À plus long terme, l’intégration de couches topologiques au sein d’architectures de type réseaux de neurones graphiques ou Transformers constitue une direction particulièrement prometteuse [8], de même que l’amélioration de la différentiabilité du calcul de persistance pour permettre un passage à l’échelle industrielle.

A Code source : Implémentation du réseau topologique

```

1 import torch
2 import torch.nn as nn
3
4 class TrueGaussianPersLay(nn.Module):
5     def __init__(self, num_gaussians=60):
6         super().__init__()
7
8         # 1. Paramètres ENTRAINABLES du PersLay
9         # Les centres t_j des gaussiennes en 2D (naissance, mort)
10        self.centers = nn.Parameter(torch.rand(num_gaussians, 2))
11
12        # Les variances (sigma) de chaque gaussienne
13        # On stocke le log de la variance pour s'assurer que la variance reste
14        # positive
15        self.log_variances = nn.Parameter(torch.zeros(num_gaussians))
16
17    def forward(self, diagrams, masks):
18        """
19        diagrams : tenseur de forme (Batch_size, Max_points, 2) avec les
20        coordonnées (birth, death).
21        masks : tenseur (Batch_size, Max_points) contenant 1 pour un vrai point
22        et 0 pour du padding.
23        """
24        # Transformation phi(p) via les gaussiennes
25        # broadcasting pour calculer la distance entre chaque point et chaque
26        # centre
27        # diagrams shape -> (B, N, 1, 2)
28        # centers shape -> (1, 1, q, 2)
29        d_exp = diagrams.unsqueeze(2)
30        c_exp = self.centers.unsqueeze(0).unsqueeze(0)
31
32        # distance au carré ||p - t_j||^2 : shape -> (B, N, q)
33        dist_sq = torch.sum((d_exp - c_exp)**2, dim=-1)
34
35        # récupération des variances : shape -> (q,)
36        variances = torch.exp(self.log_variances)
37
38        # application de la gaussienne : shape -> (B, N, q)
39        phi = torch.exp(-dist_sq / (2 * variances))
40
41        # pondération w(p)
42        # On utilise la persistance (mort - naissance) comme poids naturel
43        # diagrams[:, :, 1] = mort, diagrams[:, :, 0] = naissance
44        persistence = diagrams[:, :, 1] - diagrams[:, :, 0] # (B, N)
45        weights = persistence.unsqueeze(-1) # (B, N, 1)
46
47        # application des poids et du masque de padding
48        masks = masks.unsqueeze(-1) # (B, N, 1)
49        weighted_phi = phi * weights * masks # (B, N, q)
50
51        # Agregation (Sum Pooling)
52        # On somme sur la dimension des points (dim=1)
53        # Résultat final : shape -> (B, q)
54        out = torch.sum(weighted_phi, dim=1)
55
56        return out
57
58 class FullTopologicalNetwork(nn.Module):
59     def __init__(self):
60         super().__init__()
61         # Couche d'extraction topologique (apprend à vectoriser)

```

```
58     self.perslay = TrueGaussianPersLay(num_gaussians=60)
59
60     # classifieur standard (apprend à séparer les vecteurs)
61     self.classifier = nn.Sequential(
62         nn.Linear(60, 32),
63         nn.ReLU(),
64         nn.Linear(32, 16),
65         nn.ReLU(),
66         nn.Linear(16, 3)
67     )
68
69     def forward(self, diagrams, masks):
70         # le diagramme brut devient un vecteur de taille 60 optimisé
71         topo_features = self.perslay(diagrams, masks)
72
73         # classification
74         out = self.classifier(topo_features)
75         return out
```

Listing 1 – Fichier perslay.py

Références

- [1] Henry ADAMS et al. « Persistence images : A stable vector representation of persistent homology ». In : *Journal of Machine Learning Research* 18 (2017).
- [2] Peter BUBENIK. « Statistical topological data analysis using persistence landscapes ». In : *Journal of Machine Learning Research* 16 (2015).
- [3] Mathieu CARRIÈRE et al. « PersLay : A neural network layer for persistence diagrams and new graph topological signatures ». In : *Proceedings of Machine Learning Research* 108 (2020).
- [4] James R CLOUGH et al. « A Topological Loss Function for Deep-Learning based Image Segmentation using Persistent Homology ». In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022).
- [5] David COHEN-STEINER, Herbert EDELSBRUNNER et John HARER. « Stability of persistence diagrams ». In : *Springer* (2007).
- [6] Christoph HOFER et al. « Deep learning with topological signatures ». In : *Advances in neural information processing systems* 30 (2017).
- [7] Michael MOOR et al. « Topological autoencoders ». In : *Proceedings of Machine Learning Research* 119 (2020).
- [8] Mathilde PAPILLON et al. « Architectures of topological deep learning : A survey on topological neural networks ». In : *arXiv preprint arXiv :2304.10031* (2023).
- [9] Manzil ZAHEER et al. « Deep Sets ». In : *Advances in Neural Information Processing Systems*. 2017.